

Interrupt is coming to NYC and London this fall. Join the builders, engineers, and teams shaping what's next for agents. Get your tickets →

## LangChain Docs

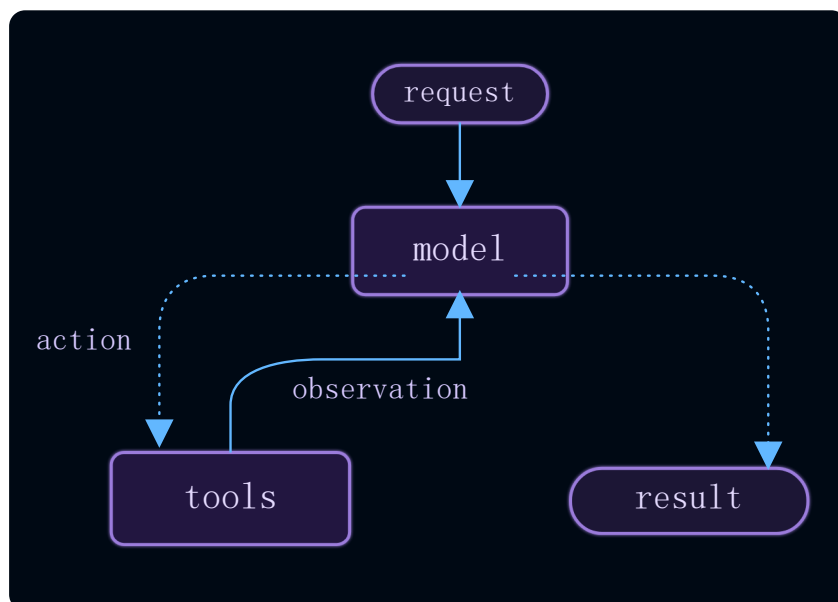
Core components > Agents

Core components

# Agents

Copy page

An agent is a model calling tools in a loop until a given task is complete.



A harness is everything around that loop: the prompt, the tools, and any middleware that shapes the model's behavior.

### ! Agent = Model + Harness

The job of a harness: get the model the right context at the right time for the given task.

`create_agent` is a highly configurable harness. At its simplest, you can create one with:

Google OpenAI Anthropic OpenRouter Fireworks Baseten Ollama

```
from langchain.agents import create_agent
```

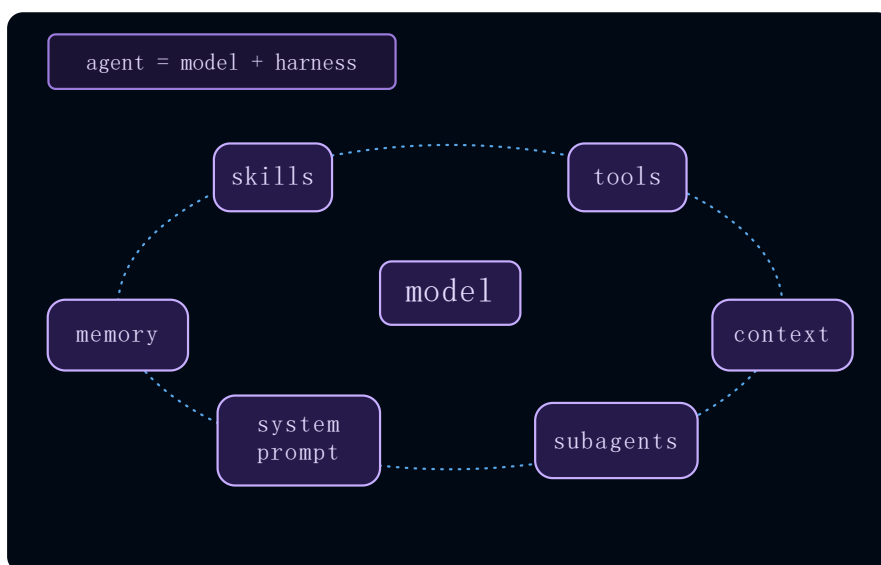
```
agent = create_agent(model="google_genai:gemini-3.6-flash", tools=tools)
```

Interrupt is coming to NYC and London this fall. Join the builders, engineers, and teams shaping what's next for agents. Get your tickets →



`create_agent` when you need to configure the harness yourself.

## Core components



## Model

Pass a model identifier string ( `"provider:model"` ) or an initialized model instance to select the model for your agent. See [Models](#) for parameters, provider setup, and dynamic model selection.

Google   OpenAI   Anthropic   OpenRouter   Fireworks   Baseten   Ollama

```
from langchain.agents import create_agent

agent = create_agent(model="google_genai:gemin-3.6-flash", tools=tools)
```

## Tools

To provide the agent with tools, pass any Python callable, LangChain tool, or tool dict. See [Tools](#) for tool definition, context access, and dynamic tool selection.

Interrupt is coming to NYC and London this fall. Join the builders, engineers, and teams shaping what's next for agents. Get your tickets →



```
def search(query: str) -> str:
    """Search for information."""
    return f"Results for: {query}"

agent = create_agent(model="google_genai:gemini-3.6-flash", tools=[search])
```

## System prompt

Shape how the agent approaches tasks. The system prompt parameter accepts a string or `SystemMessage` . For dynamic prompts at runtime, use [middleware](#).

Google OpenAI Anthropic OpenRouter Fireworks Baseten Ollama



```
agent = create_agent(
    model="google_genai:gemini-3.6-flash",
    tools=tools,
    system_prompt="You are a helpful assistant. Be concise and accurate.",
)
```

## Structured output

Return a validated schema from the agent using `response_format=` . See [Structured output](#) for strategies and examples.

Interrupt is coming to NYC and London this fall. Join the builders, engineers, and teams shaping what's next for agents. Get your tickets →



```
summary: str
confidence: float
```

```
agent = create_agent(model="google_genai:gemin-3.6-flash", tools=tools, response_schema=StructuredResponse)
result = agent.invoke({"messages": [{"role": "user", "content": "Summarize AI agents in 100 words."}]}
result["structured_response"]  # Answer(summary=..., confidence=...)
```

## Agent state

Every agent manages its execution context through `AgentState`, a typed dictionary that holds the current conversation history and any custom fields your tools and middleware need.

The built-in field is:

| Field                 | Type                           | Description                                                                                                |
|-----------------------|--------------------------------|------------------------------------------------------------------------------------------------------------|
| <code>messages</code> | <code>list[BaseMessage]</code> | The full conversation history for the current thread. Append-only: new messages are added, never replaced. |

`AgentState` is also the type signature for every node-style middleware hook (`before_model`, `after_model`, and similar). Hooks receive the current state and can return a dict of updates to merge back into it.

To add custom fields (for example, a `user_id` or a counter), subclass `AgentState` and pass the subclass to `create_agent` via `state_schema=`:

Interrupt is coming to NYC and London this fall. Join the builders, engineers, and teams shaping what's next for agents. Get your tickets →

## LangChain Docs

```
call_count: int

agent = create_agent(
    model="google_genai:gemini-3.6-flash",
    tools=[],
    state_schema=MyState,
)
```

For full details, examples, and middleware-level state schemas, see [Short-term memory](#) and [Custom middleware](#).

## Invocation

💡 Trace each step of this loop, debug tool calls, and evaluate agent outputs with [LangSmith](#). Follow the [tracing quickstart](#) to get set up. We recommend you also set up [LangSmith Engine](#) which monitors your traces, detects issues, and proposes fixes.

You can invoke an agent with a message. Behind the scenes that passes an update to the agent's `state`. All agents include a [sequence of messages](#) in their state; to invoke the agent, pass a new message along with a `thread_id` so the agent can persist and resume conversation history:

Interrupt is coming to NYC and London this fall. Join the builders, engineers, and teams shaping what's next for agents. Get your tickets →

## LangChain Docs

```
model="google_genai:gemini-3.6-flash",
tools=[],
checkpointer=InMemorySaver(),
)

config = {"configurable": {"thread_id": str(uuid7())}}

result = agent.invoke(
    {"messages": [{"role": "user", "content": "What's the weather in San Francisco?"}],
    config=config,
)

# A follow-up turn on the same conversation: reuse the same thread_id to keep the history
result = agent.invoke(
    {"messages": [{"role": "user", "content": "What about tomorrow?"}]},
    config=config,
)
```

### View example trace



Open a public LangSmith run for this example.

❗ Persisting conversation history with `thread_id` requires the agent to be configured with a checkpointer. When deployed on LangSmith, a checkpointer is provisioned automatically. Locally, pass one explicitly, for example `create_agent(..., checkpointer=InMemorySaver())`.

If you also need to pass per-run configuration (such as a user ID, API keys, or feature flags) to tools and middleware, pass it as `context` alongside `config`. Define the shape of that data with `context_schema` and access it through `runtime.context`:

Interrupt is coming to NYC and London this fall. Join the builders, engineers, and teams shaping what's next for agents. Get your tickets →



```
@dataclass
class Context:
    user_id: str

agent = create_agent(
    model="google_genai:gemini-3.6-flash",
    tools=[],
    context_schema=Context,
    checkpointer=InMemorySaver(),
)

result = agent.invoke(
    {"messages": [{"role": "user", "content": "What's the weather in San Francisco?"}],
    config={"configurable": {"thread_id": str(uuid7())}},
    context=Context(user_id="user-123"),
)
```

**View example trace**



Open a public LangSmith run for this example.

`thread_id` scopes the *conversation* (message history, checkpoints), while `context` carries *per-run* data your tools and middleware read at invocation time. Both are commonly passed together. See [tool context](#) and [Runtime](#) for more.

## Streaming

`invoke` returns the final response at the end of a run. If an agent executes multiple tool calls, users often need progress updates before completion. Use streaming to surface intermediate messages and tool activity as they happen.

Interrupt is coming to NYC and London this fall. Join the builders, engineers, and teams shaping what's next for agents. Get your tickets →



```
)  
for snapshot in stream.values:  
    # Each snapshot contains the full state at that point  
    latest_message = snapshot["messages"][-1]  
    if latest_message.content:  
        if isinstance(latest_message, HumanMessage):  
            print(f"User: {latest_message.content}")  
        elif isinstance(latest_message, AIMessage):  
            print(f"Agent: {latest_message.content}")  
    elif latest_message.tool_calls:  
        print(f"Calling tools: {[tc['name'] for tc in latest_message.tool_calls]})
```

### View example trace



Open a public LangSmith run for this example.



For streaming modes, event types, and UI patterns, see [Streaming](#).

## Configure the harness

`create_agent` is highly extensible. Middleware is the primitive for customization: each piece handles one concern, hooks into the agent loop at the right moment, and composes freely with any other. Take exactly what your use case needs and skip the rest.

Common patterns are prebuilt as first-class middleware. You can build anything else as [custom middleware](#).

Interrupt is coming to NYC and London this fall. Join the builders, engineers, and teams shaping what's next for agents. Get your tickets →



SKILLS

memory

Summarization

Context offloading

Prompt caching

## CONTEXT MANAGEMENT

As agents take on complex work, they need support across a few key areas. The middleware ecosystem provides:

**Execution environment**

Tools, filesystem, sandboxes, and code execution

**Context management**

Summarization, memory, skills, and prompt caching

**Planning and delegation**

Todo lists and subagents for parallel, isolated work

**Fault tolerance**

Retries, fallbacks, and call limits

**Guardrails**

PII detection and content controls

**Steering**

Human-in-the-loop approval before high-impact actions



`create_deep_agent` pre-assembles this stack for long-running coding and research tasks (filesystem, summarization, subagents, and prompt caching included by default). See [Deep Agents](#) for the full prebuilt harness.

Interrupt is coming to NYC and London this fall. Join the builders, engineers, and teams shaping what's next for agents. Get your tickets →

## LangChain Docs

Google OpenAI Anthropic OpenRouter Fireworks Baseten Ollama



```
from langchain.agents import create_agent
from deepagents.backends import StateBackend
from deepagents.middleware import FilesystemMiddleware

agent = create_agent(
    model="google_genai:gemini-3.6-flash",
    tools=[search],
    middleware=[FilesystemMiddleware(backend=StateBackend())],
)
```

See [FilesystemMiddleware](#), [Sandboxes](#), [Interpreters](#).

ⓘ This example imports from the `deepagents` package. Install it with:

`pip` `uv`



```
pip install deepagents
```

## Context management

Every model call has a fixed context window. As an agent runs, that window fills with accumulating history, tool results, and intermediate steps. Summarization compresses history before overflow hits; memory loads persistent instructions at startup so knowledge carries across sessions; skills surface domain knowledge on demand rather than loading everything upfront.

Interrupt is coming to NYC and London this fall. Join the builders, engineers, and teams shaping what's next for agents. Get your tickets →

## LangChain Docs

```
agent = create_agent(  
    model=model,  
    tools=[search],  
    middleware=[  
        FileSystemMiddleware(backend=backend),  
        SummarizationMiddleware(model=model, backend=backend),  
        MemoryMiddleware(backend=backend, sources=["./AGENTS.md"]),  
        SkillsMiddleware(backend=backend, sources=["./skills/"]),  
    ],  
)
```

See [SummarizationMiddleware](#), [MemoryMiddleware](#), [Skills](#), [Context engineering](#).

ⓘ This example imports from the `deepagents` package. Install it with:

```
pip uv  
—  
pip install deepagents
```



## Planning and delegation

Complex tasks often exceed what one context window can handle. Delegation lets the main agent break work into pieces, hand them to subagents that each run in their own isolated context, and stay focused on coordination rather than execution. Work can run in parallel; the main agent's context stays clean.

Interrupt is coming to NYC and London this fall. Join the builders, engineers, and teams shaping what's next for agents. Get your tickets →



```
from langchain.tools import tool

@tool
def search(query: str) -> str:
    """Search for a query and return a short summary."""
    return f"Search results for: {query}"

backend = StateBackend()

agent = create_agent(
    model="google_genai:gemini-3.6-flash",
    tools=[search],
    middleware=[
        FileSystemMiddleware(backend=backend),
        TodoListMiddleware(),
        SubAgentMiddleware(
            backend=backend,
            subagents=[
                {
                    "name": "researcher",
                    "description": "Searches and returns a structured summary.",
                    "system_prompt": "Use the search tool to research the ques",
                    "tools": [search],
                    "model": "anthropic:claude-sonnet-4-6",
                    "middleware": [],
                }
            ],
        ),
    ],
)
```

See [Subagents](#).

Interrupt is coming to NYC and London this fall. Join the builders, engineers, and teams shaping what's next for agents. Get your tickets →



## Name your agent

Optionally use an identifier for the agent. This is especially useful when embedding the agent as a subgraph in multi-agent systems.

Google OpenAI Anthropic OpenRouter Fireworks Baseten Ollama



```
agent = create_agent(model="google_genai:gemini-3.6-flash", tools=tools, name=
```

## Fault tolerance

Agents in production encounter failures that rarely appear in development: rate limits, model timeouts, transient API errors. Fault tolerance middleware handles these at the infrastructure level so your tools and business logic don't need try/catch around every call.

Interrupt is coming to NYC and London this fall. Join the builders, engineers, and teams shaping what's next for agents. Get your tickets →



```
@tool
def search(query: str) -> str:
    """Search for a query and return a short summary."""
    return f"Search results for: {query}"

agent = create_agent(
    model="google_genai:gemini-3.6-flash",
    tools=[search],
    middleware=[
        ModelRetryMiddleware(max_retries=3),
        ToolRetryMiddleware(max_retries=2),
    ],
)
```

See [ModelRetryMiddleware](#), [ToolRetryMiddleware](#), [Prebuilt middleware](#).

## Guardrails

Some policies can't live in a prompt—they need to be enforced deterministically regardless of what the model does. Guardrails intercept data as it flows through the agent loop, applying compliance rules or content policies before tool results reach the model's context.

Interrupt is coming to NYC and London this fall. Join the builders, engineers, and teams shaping what's next for agents. Get your tickets →



```
@tool
def search(query: str) -> str:
    """Search for a query and return a short summary."""
    return f"Search results for: {query}"

agent = create_agent(
    model="google_genai:gemini-3.6-flash",
    tools=[search],
    middleware=[PIIMiddleware("email")],
)
```

See [PIIMiddleware](#), [Prebuilt middleware](#).

## Steering

Full autonomy isn't always appropriate. Steering lets you place humans at specific decision points—before destructive writes, expensive API calls, or anything requiring judgment—without restructuring your agent. The agent pauses and waits; a human approves, edits, or rejects; execution continues.

Interrupt is coming to NYC and London this fall. Join the builders, engineers, and teams shaping what's next for agents. Get your tickets →



```
@tool
def search(query: str) -> str:
    """Search for a query and return a short summary."""
    return f"Search results for: {query}"

agent = create_agent(
    model="google_genai:gemini-3.6-flash",
    tools=[search],
    middleware=[HumanInTheLoopMiddleware(interrupt_on={"write_file": True})],
)
```

See [HumanInTheLoopMiddleware](#) , [Human-in-the-loop](#).

## Middleware resources



### Middleware overview

How the middleware stack works and when hooks fire



### Prebuilt middleware

Full reference with configuration examples



### Custom middleware

Write your own hooks for business logic, PII scrubbing, and more



[Connect these docs](#) to Claude, VSCode, and more via MCP for real-time answers.



[Edit this page on GitHub](#) or [file an issue](#).

Interrupt is coming to NYC and London this fall. Join the builders, engineers, and teams shaping what's next for agents. Get your tickets →



---

# LangChain Docs

## Resources

[Forum](#)

[Changelog](#)

[LangChain Academy](#)

[Contact Sales](#)

## Company

[Home](#)

[Trust Center](#)

[Careers](#)

[Blog](#)